



UNIVERSITY OF MALAYA

TECHNICAL REPORT

WIX1002 FUNDAMENTALS OF PROGRAMMING

TOPIC 2 - LEDGER SYSTEM

OCC : 1

GROUP : 3

LECTURER NAME : DR UNAIZAH HANUM BINTI OBAIDELLAH

GROUP MEMBERS:

NAME	MATRICS NUMBER
MUJAHID BIN ABDULLAH OMAR WAHBI	24063585
MD MUNTASIR UL HAKIM	23082428
MUHAMMAD FAKHRUL BIN HASSAN	23002457
IRFAN IZZUDDIN BIN AZMI	23002562
SIMON ROY	24004521

CSV Handler

1. Assigned Project

The **CSVHandler** class is designed to handle file management tasks within a larger ledger or data processing system. It ensures the existence of necessary CSV files and facilitates data handling by providing a method to count the lines in a CSV file. This utility simplifies file setup and validation processes, ensuring the system operates efficiently.

2. Requirements

The requirements derived from the code are:

1. Ensure the existence of CSV files: The program should check if specific CSV files exist. If they do not, it creates them automatically.
2. Count the number of lines in a CSV file: Provide a method to determine how many lines a given CSV file contains. This feature is useful for processing and analyzing data stored in these files.

3. Approach to Solve It:

Methods Implemented:

- ensureCSVFiles(String... filePaths):

- Accepts a list of file paths.
- Iterates through each path to check if the file exists.
- If a file does not exist, it creates an empty file.

- countLines(String filePath):

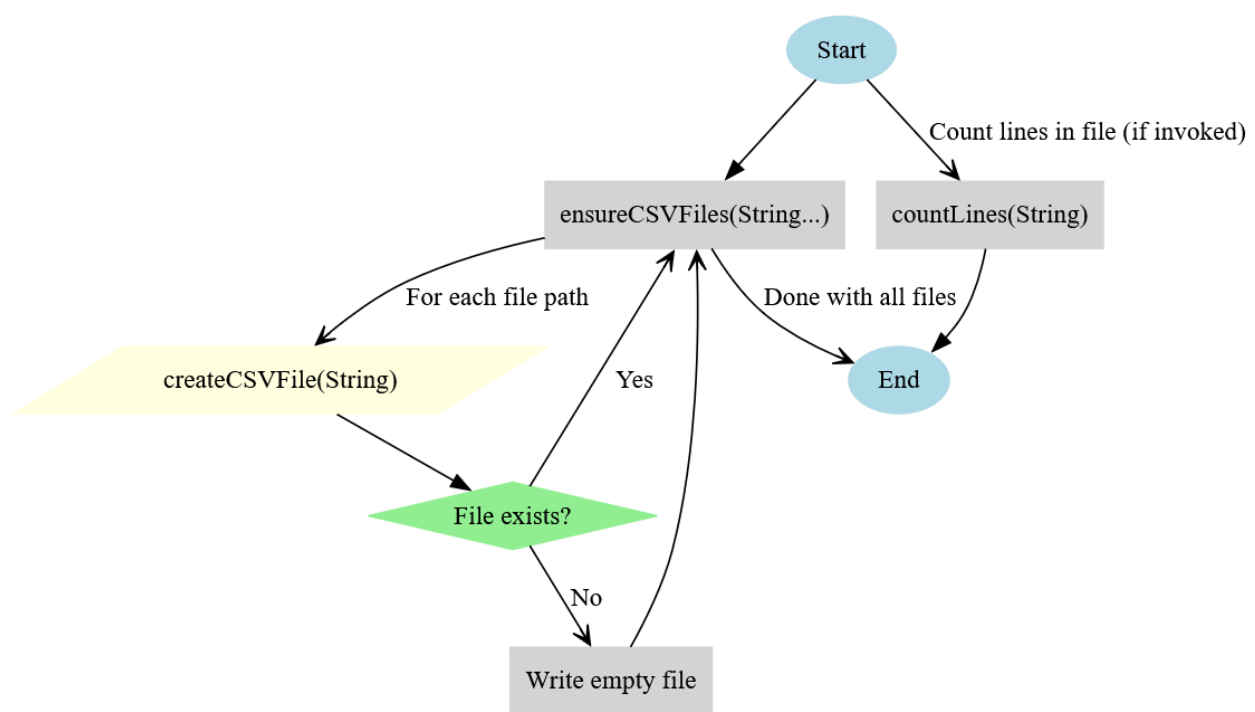
- Takes a single file path as input.
- Uses the `Files.lines` method to read the file and count the number of lines.

Libraries Used:

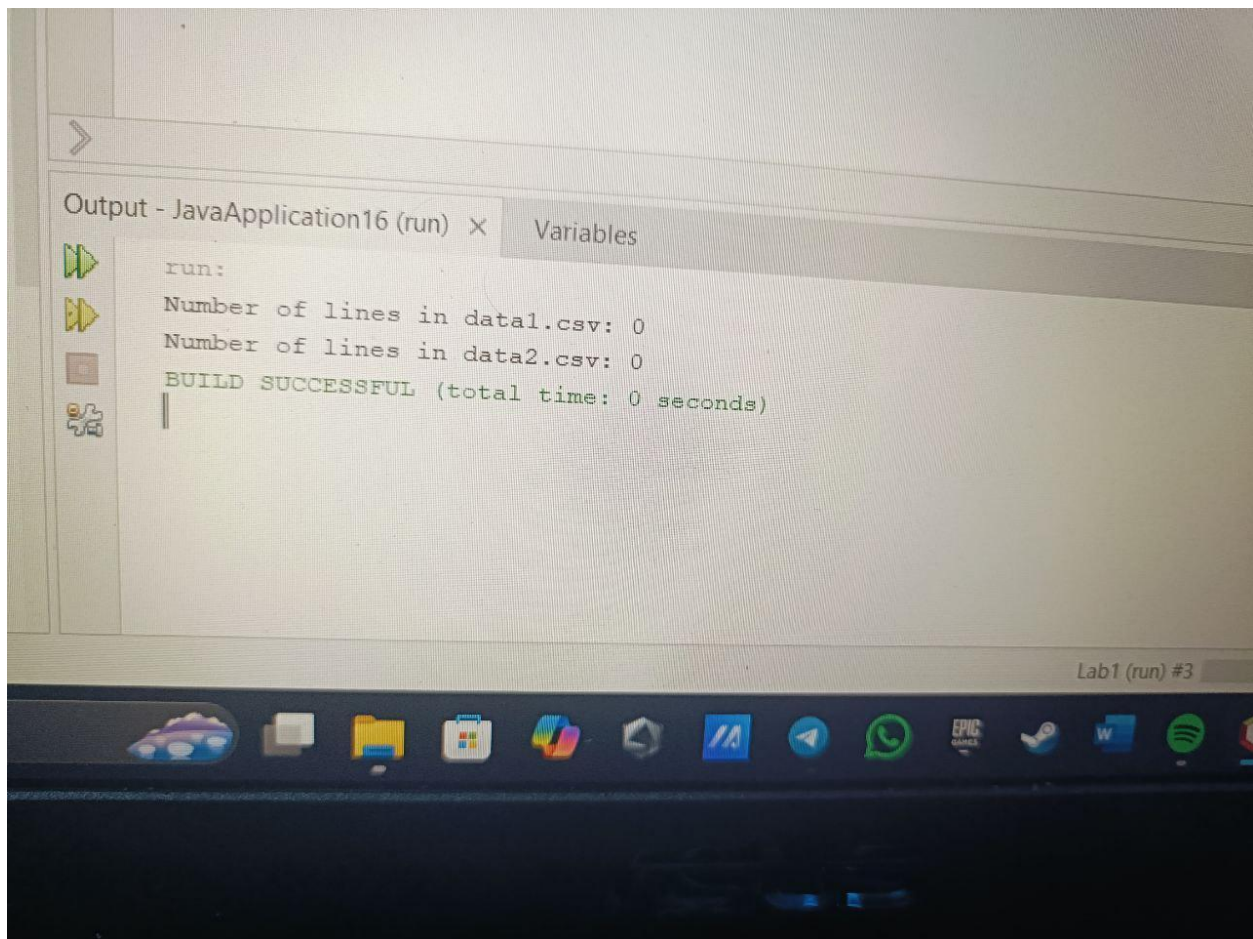
- `java.io.*`: For handling basic input/output operations.
- `java.nio.file.*`: Provides modern utilities for file handling, such as checking file existence, writing files, and reading lines efficiently.

4. Flow Chart

The following flow chart describes the process:



Output:



Data Models

1. Assigned Project

The `DataModels` class provides structured representations of financial data, including spending, savings, and loan information. These models are designed to standardize how data is stored and accessed in a ledger or financial tracking system.

2. Requirements

From the file content, the requirements include:

- 1. Spending Data Model:**
 - Represent spending with attributes for date and amount.
 - Provide access methods to retrieve the date and amount.
- 2. Savings Data Model:**
 - Represent savings with attributes for date and total savings.
 - Provide access methods to retrieve the date and total savings.
- 3. Loan Data Model:**
 - Represent loan repayments with attributes for date and repayment amount.
 - Provide access methods to retrieve the date and repayment amount.
- 4. Loan Repayment Data Model:**
 - Represent loan repayments with date and amount.
 - Provide access methods to retrieve this data.

3. Approach to Solve It:

Methods Implemented:

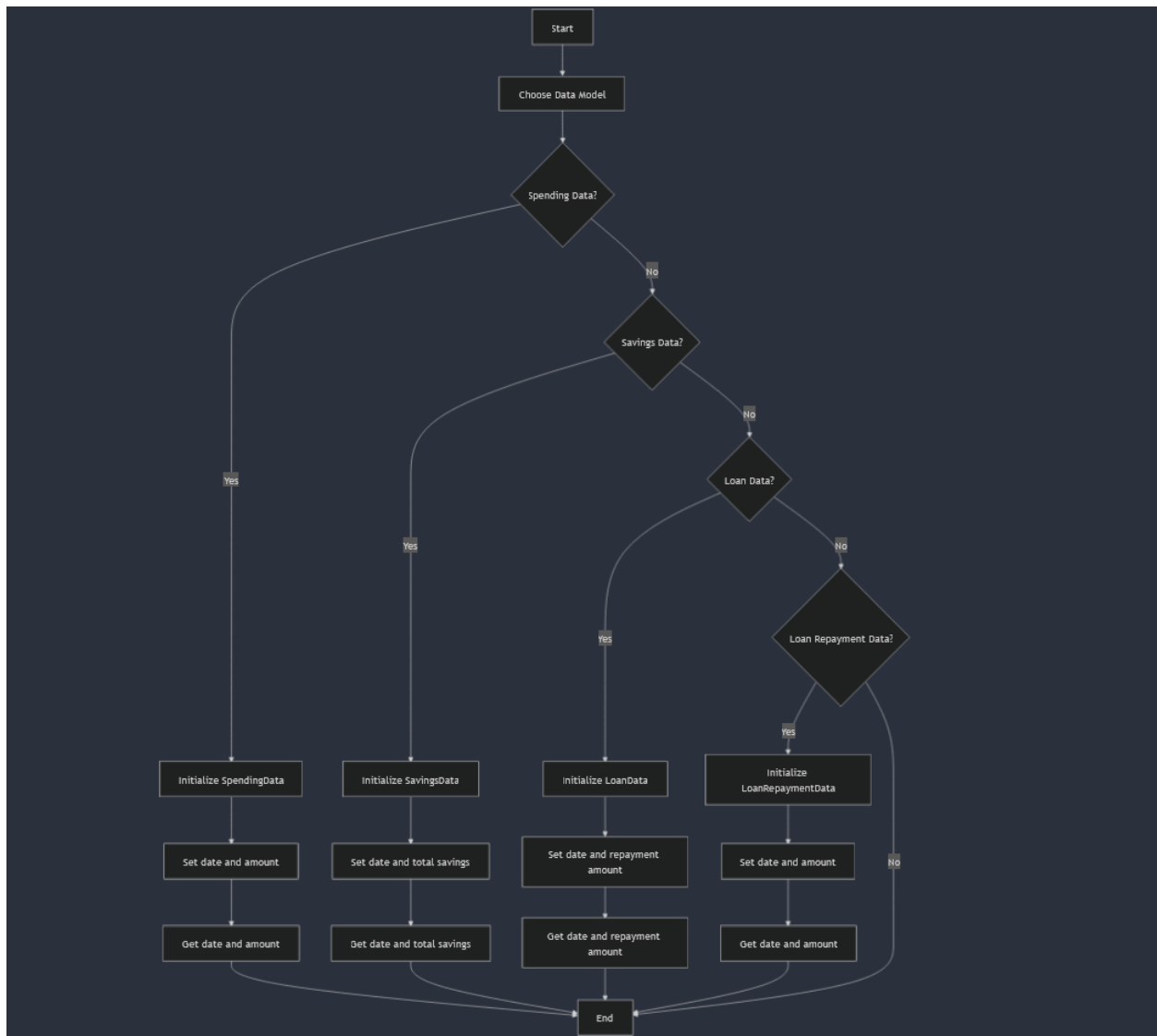
- **Encapsulation:** Each data model is implemented as a static inner class with private fields for encapsulation.

- **Constructors:** Parameterized constructors initialize the data fields.
- **Getter Methods:** Public getter methods (`getDate`, `getAmount`, etc.) allow controlled access to the private fields.

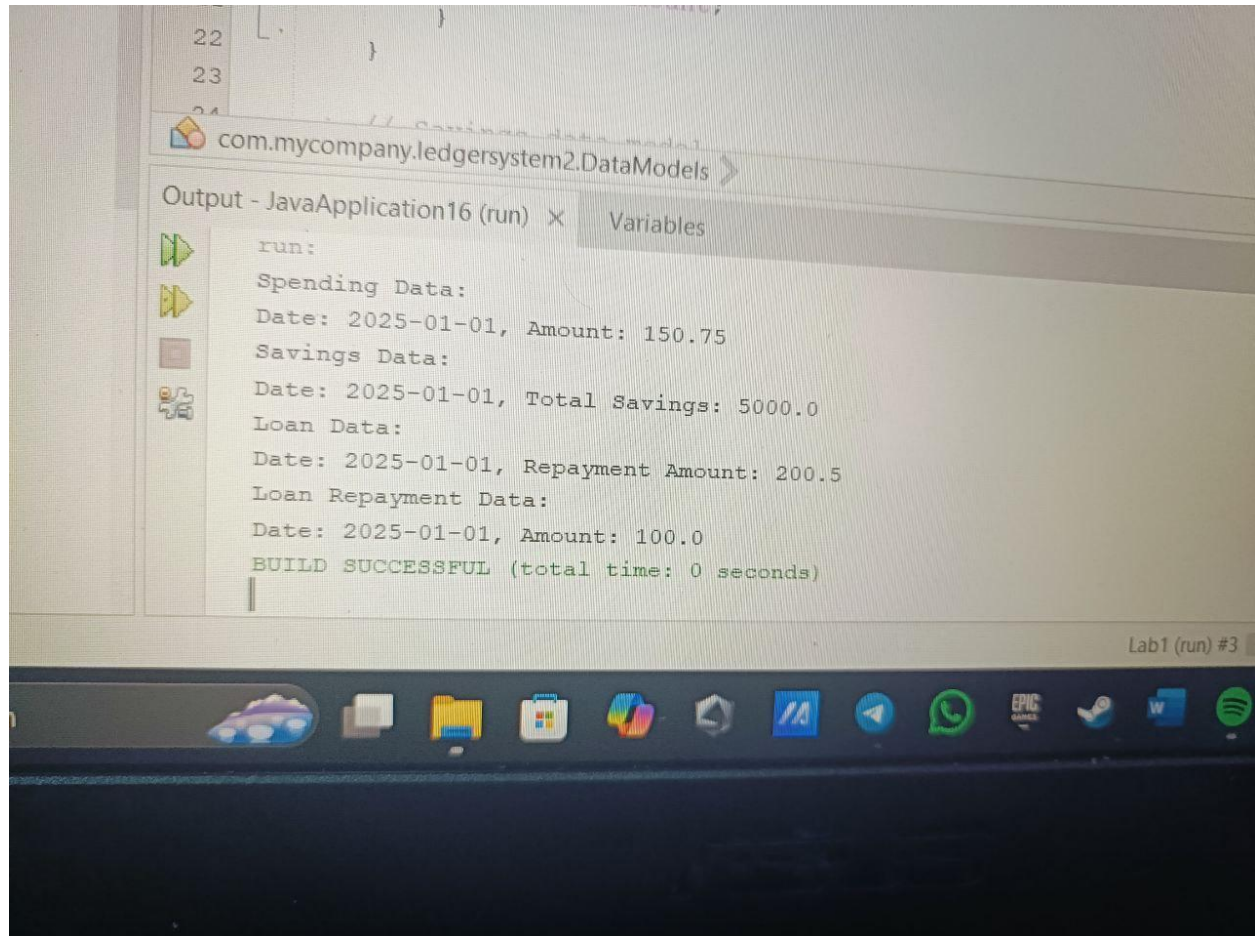
Libraries Used:

- The class relies solely on core Java (`java.lang`), as there are no imports for external libraries.

4. Flow Chart



5. Sample Snapshot of Output



Data Visualizer

1. Assigned Project

‘DataVisualizer’ this class is responsible for displaying spending trends, savings growth, and loan repayments over time using graphs or charts.

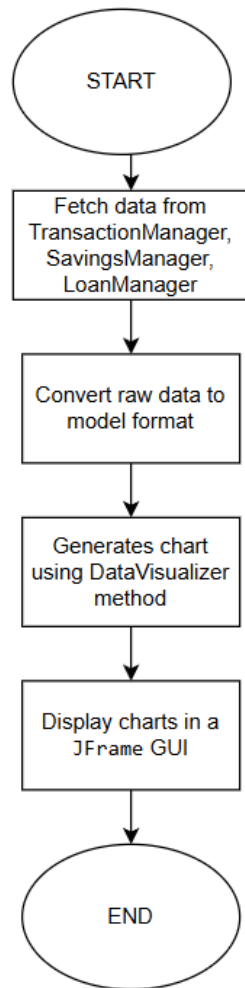
2. Requirements

- Create spending trend, savings growth and loan repayments chart based on user’s transaction.
- Fetch the data from transaction managers ([TransactionManager](#), [SavingsManager](#), and [LoanManager](#)).
- Render the data in a JFrame window with charts displayed using [ChartPanel](#).

3. Approach to solve the problem

- **Data Acquisition:**
 - Use [TransactionManager](#), [SavingsManager](#), and [LoanManager](#) to fetch financial data.
- **Visualization:**
 - Implement methods in [DataVisualizer](#) to create different types of charts:
 - Line charts using [DefaultCategoryDataset](#) and [ChartFactory](#).
 - Pie charts using [DefaultPieDataset](#) and [ChartFactory](#).
 - Use [ChartPanel](#) to embed the charts into the application.
- **Display:**
 - Create a [JFrame](#) to hold and organize the chart panels using [BoxLayout](#).

4. Flow Chart



5. Sample snapshot of output



Interest Predictor

1. Assigned Project

The InterestPredictor class is part of a ledger system used to calculate and predict interest earnings for users based on their choices of banks, deposit amount and duration.

2. Requirements

- Calculate interest earned over a specified period (e.g., daily, monthly, or annually).
- The predictor is just to show the earned interest on their balance based on the bank interest
- Display the calculated interest rate in a readable format

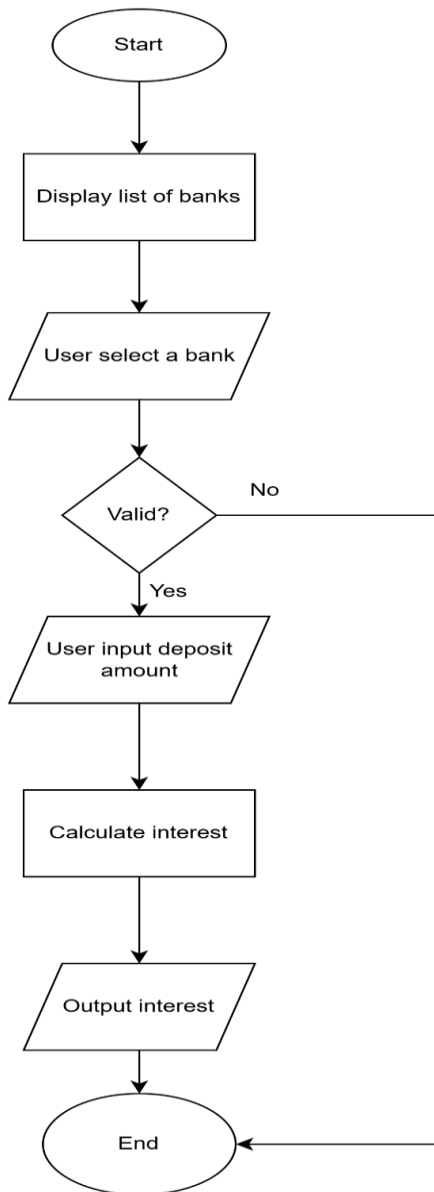
3. Approach to solve the problem

- Switch Expressions: Used for selecting interest rates based on the user's bank choice.
- Input Handling: Java's **Scanner** is used to gather inputs from the user.
- Arithmetic Calculations: Basic mathematical formulas for interest calculation.
- Validation: Ensures that invalid choices do not proceed with calculations.

Key Steps:

1. Display a menu of available banks with their corresponding interest rates.
2. Use a switch expression to assign the interest rate based on the user's choice.
3. Validate the user inputs for deposit amount and duration.
4. Calculate the monthly and total interest.
5. Display the calculated results.

4. Flow Chart



5. Sample snapshot of output

Valid input:

```
Available Banks:
1. RHB (2.6%)
2. Maybank (2.5%)
3. Alliance (2.85%)
4. AmBank (2.55%)
5. Standard Chartered (2.65%)
Choose a bank: 1
Enter deposit amount: 10000
Enter duration in months: 12
Predicted interest for 12 months: 260.00
```

Invalid Input:

```
Available Banks:
1. RHB (2.6%)
2. Maybank (2.5%)
3. Alliance (2.85%)
4. AmBank (2.55%)
5. Standard Chartered (2.65%)
Choose a bank: 6
Invalid choice!
```

Ledger System

1. Assigned Project

The LedgerSystem class is the main entry point for the whole ledger management application. It facilitates user registration, login, and a range of financial management functionalities, including recording transactions, handling savings, loans, and predicting deposit interest

2. Requirements

1. User Management:

- Register users.
- Authenticate users during login.

2. Financial Transactions:

- Record and categorize transactions (debits and credits).
- Maintain transaction history with filtering and sorting options.
- Export transaction history to external files (CSV).

3. Savings Management:

- Automate savings transfers.
- Display savings balance.

4. Loan Management:

- Handle loans, repayments, and send repayment reminders.

5. Prediction Tools:

- Calculate interest on deposits.

6. Visualization:

- Display financial data as graphs using [JFreeChart](#).

7. File Management:

- Use CSV files for data persistence.

3. Approach to solve problem

- **User Registration and Login:**

- a. Validate and store user credentials.
- b. Provide unique user IDs.

- **Dashboard:**

- c. Central hub for accessing all application features.
- d. Continuously loop until the user logs out.

- **Transaction Handling:**

- e. Record and classify debit/credit transactions.
- f. Maintain history in CSV.

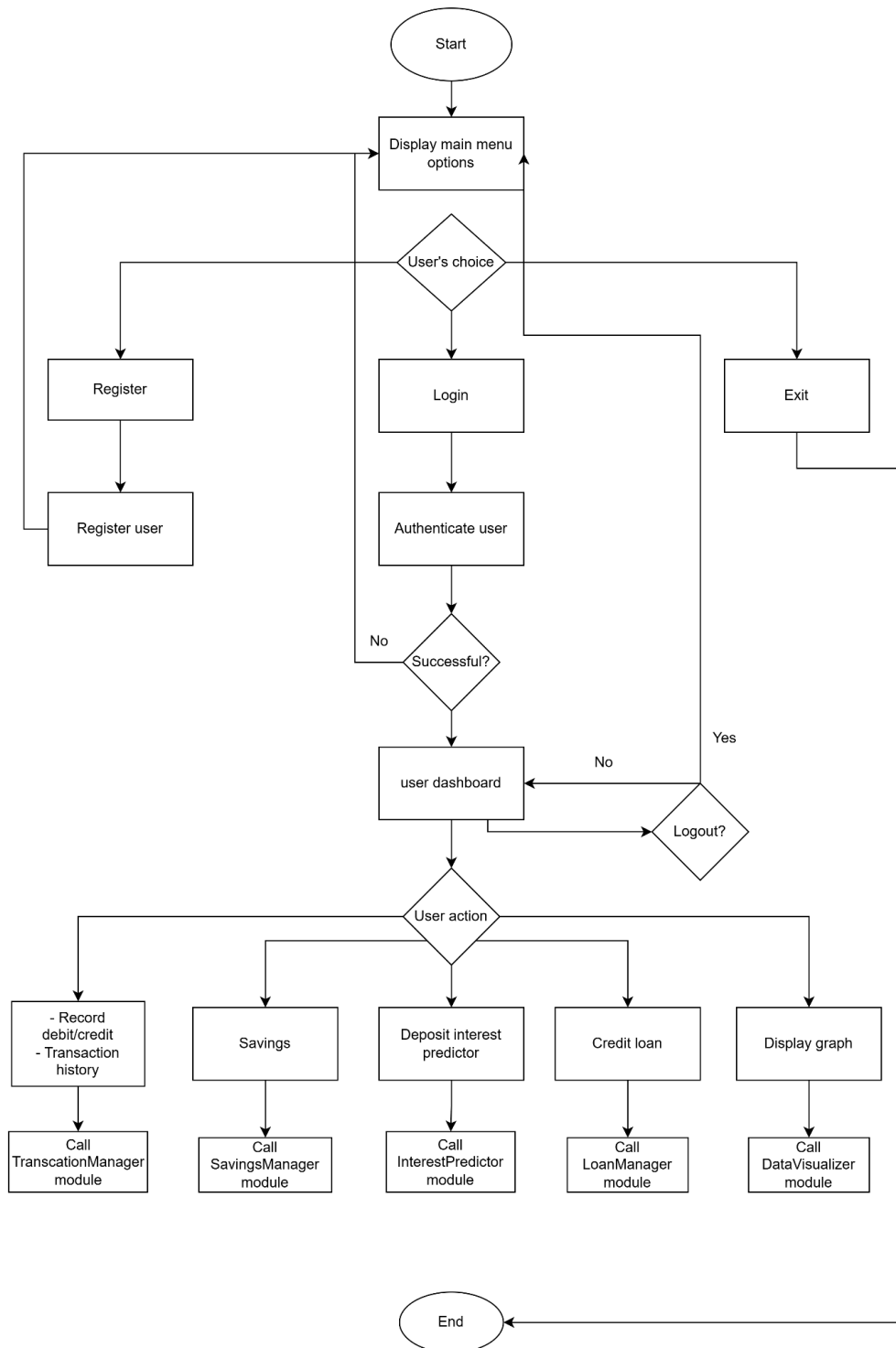
- **Savings and Loans:**

- g. Automate certain functionalities (e.g., auto-savings).
- h. Implement predictive algorithms for interest and repayment schedules.

- **Exporting:**

- i. Generate CSV files for transaction history.

4. Flow Chart



5. Sample Snapshot of output

```
Welcome to the Ledger System!
1. Register
2. Login
3. Exit
Choose an option: 2

Enter username: Abu
Enter password: *****

Welcome, Abu ==
Balance: $1000.00
Savings: $500.00
Loan: $200.00

Main Menu:
1. Record Debit
2. Record Credit
3. Savings
4. View Transaction History
5. Filter & Sort Transaction History
6. Export Transaction History
7. Deposit Interest Predictor
8. Credit Loan
9. Display graph
10. Logout
Choose an option:
```


Loan Manager

1. Assigned project

The Loan Manager class is typically a service class in object-oriented programming. The purpose of the Loan Manager class in a ledger system is to manage loan transactions, track balances, calculate interest, monitor repayment schedules, and ensure compliance with loan terms.

2. Requirements

- **1. Loan Data Management**

- § **Loan Details:** Store information like loan amount, interest rate, loan term, repayment schedule, etc.

- § **Transaction History:** Maintain records of loan transactions such as disbursements, repayments, and interest payments.

- **2. Interest Calculation**

- § **Interest Rate:** Apply the appropriate interest rate (fixed or variable) based on the loan type and terms.

- § **Interest Calculation Logic:** Implement methods to calculate interest over time, including daily, monthly, or yearly calculations.

- **3. Repayment Schedule**

- § **Due Dates:** Track repayment dates and amounts due based on loan terms.

- § **Payment Application:** Apply repayments correctly to the principal and interest.

- § **Late Fees:** Calculate and apply late fees if repayments are missed or delayed.

· 4. Account Balances and Ledger Updates

§ **Balance Management:** Ensure accurate tracking of outstanding balances for each loan.

§ **Ledger Integration:** Update the financial ledger system when transactions occur (e.g., loan disbursements, repayments).

3. Approach to solve the problem

1. User Interaction and Choice Handling

- **Menu Options:** The system starts by offering the user two options:
 - **Apply for Loan:** If the user does not have an active loan, they can apply for a new loan.
 - **Repay Loan:** If the user has an active loan, they can repay it by entering a repayment amount.

The user's choice is captured and processed via a Scanner object.

2. Checking Active Loans (hasActiveLoan)

- **Active Loan Check:** The function hasActiveLoan reads a CSV file (loans.csv) containing loan details to check whether a user has an active loan. It loops through the file, skipping the header, and checks if the user has an active loan based on their user ID and the status field ("active").

3. Loan Application (applyForLoan)

- **Loan Application:** If the user does not have an active loan, they can apply for a new loan by providing:
 - Principal amount
 - Annual interest rate
 - Repayment period in months

- The total repayment amount is calculated based on these inputs. The loan is then saved to the loans.csv file, and the loan status is marked as "active".

4. Loan Repayment (repayLoan)

- Repaying Loans: If the user has an active loan, the system:
 - Displays the outstanding balance.
 - Prompts the user for a repayment amount.
 - Updates the loan details, either reducing the balance or marking the loan as "repaid" if the loan is fully paid off.
 - A temporary file is used to update the loan information, which is then renamed back to the original file after successful repayment.
- Repayment details are also recorded in a separate file (loansRepayment.csv).

5. Displaying Loan Balance (displayLoan)

- Display Current Loan Balance: This method reads the loans.csv file to find the user's loan and displays the current balance.

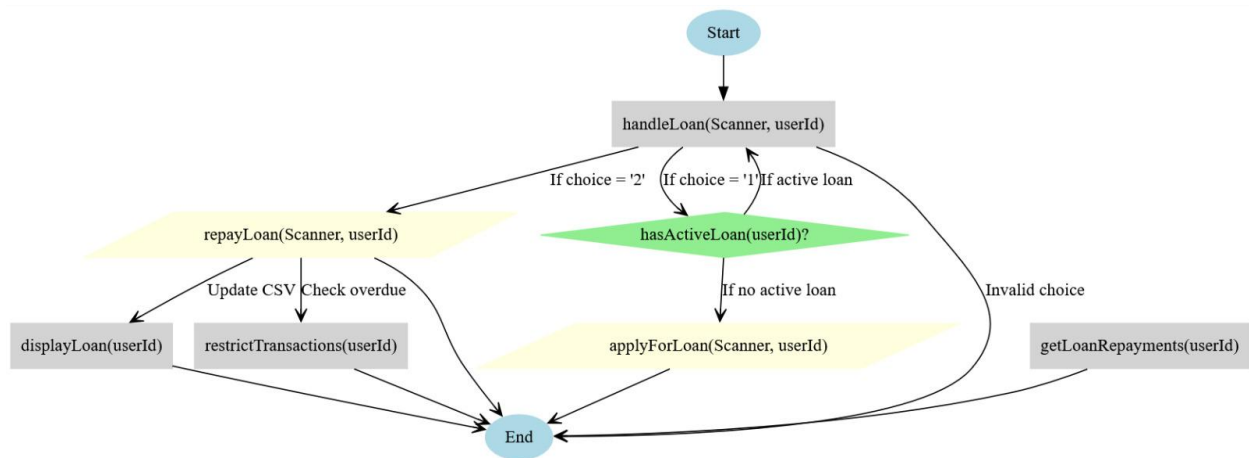
6. Transaction Restrictions (restrictTransactions)

- Transaction Restrictions for Overdue Loans: Before any new transactions are allowed, the system checks if the user has an overdue loan. If the due date is passed, transactions are restricted. The due date is determined from the loan entry in the loans.csv file.

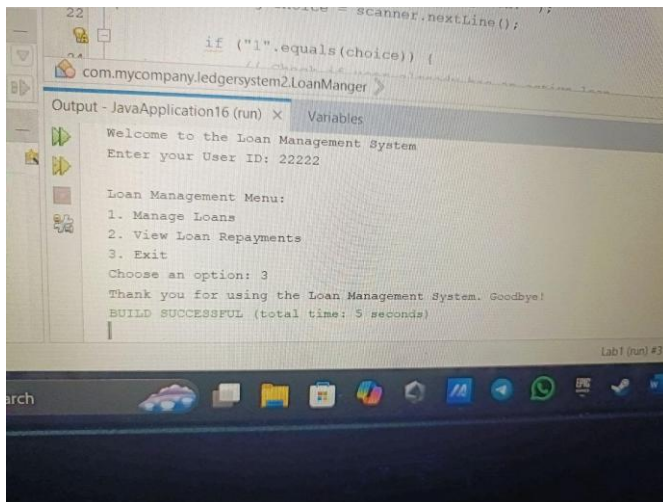
7. Loan Repayments Tracking (getLoanRepayments)

- Tracking Repayments: This method retrieves all repayment records for a specific user from the loansRepayment.csv file, summing up the repayments made on different dates and returning them in a map with dates as keys and repayment amounts as values.

4. Flowchart



5. Sample Output



Reminder System

1. Assigned project

The Reminder System class is a utility class in Java. This class is responsible for checking loan repayment dates and sending reminders to users about upcoming repayments.

2. Requirements

1. Reading Loan Data:

- The ReminderSystem class must be able to read loan information from a data source (e.g., a CSV file) to retrieve details about loans such as the loan ID, user ID, and due date.

2. Identifying the User's Loans:

- The system should accept a userId and filter loans belonging to that specific user.
- For each loan associated with the user, the due date must be checked to determine if it falls within the next 30 days.

3. Sending Repayment Reminders:

- The class should identify loans due within the next 30 days (from the current date).
- It should print a reminder message for each loan due within this time frame, including:
 - o Loan ID
 - o Due date
 - o Number of days left until repayment

- If no loans are due within the next 30 days, the system should notify the user that there are no upcoming repayments.

3. Approach to solve the problem

Step 1: Read the Loan Data

- Open the CSV file using `BufferedReader`.
- Loop through each line, split it by commas, and extract loan details like loan ID, user ID, and due date.
- Filter the loans based on the provided `userId`.

Step 2: Calculate the Date Range

- Get today's date using `LocalDate.now()`.
- Calculate the reminder threshold as `today.plusDays(30)`.

Step 3: Filter and Calculate Loan Due Dates

- Loop through the filtered loans for the specific user.
- Compare the due date of each loan with the current date and the reminder threshold (30 days).
- For loans due within the next 30 days, calculate the number of days left until repayment using `ChronoUnit.DAYS.between(today, dueDate)`.

Step 4: Send Reminders

- For each loan due within the next 30 days, print a reminder message.
- If no loans are due within the next 30 days, print a message saying "No upcoming repayments within the next 30 days."

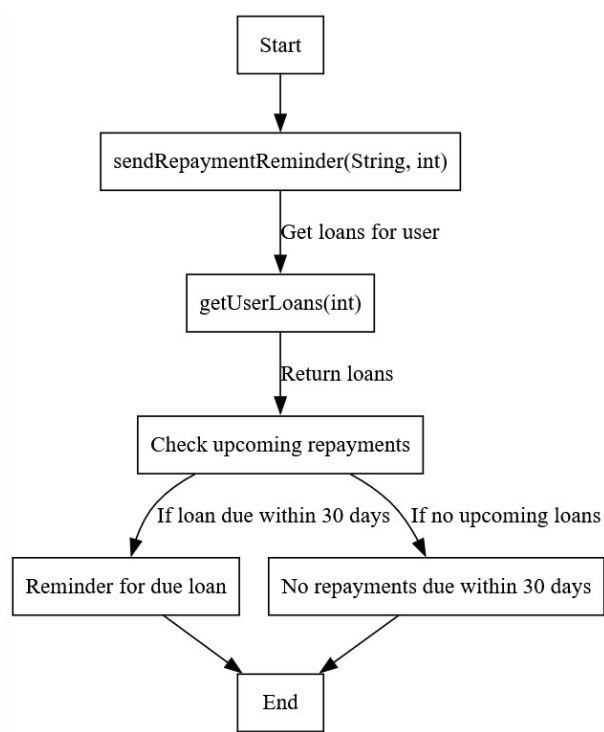
Step 5: Error Handling

- Handle errors in reading the CSV file (e.g., file not found, I/O errors).
- Handle invalid loan data such as malformed dates by catching `DateTimeParseException`.

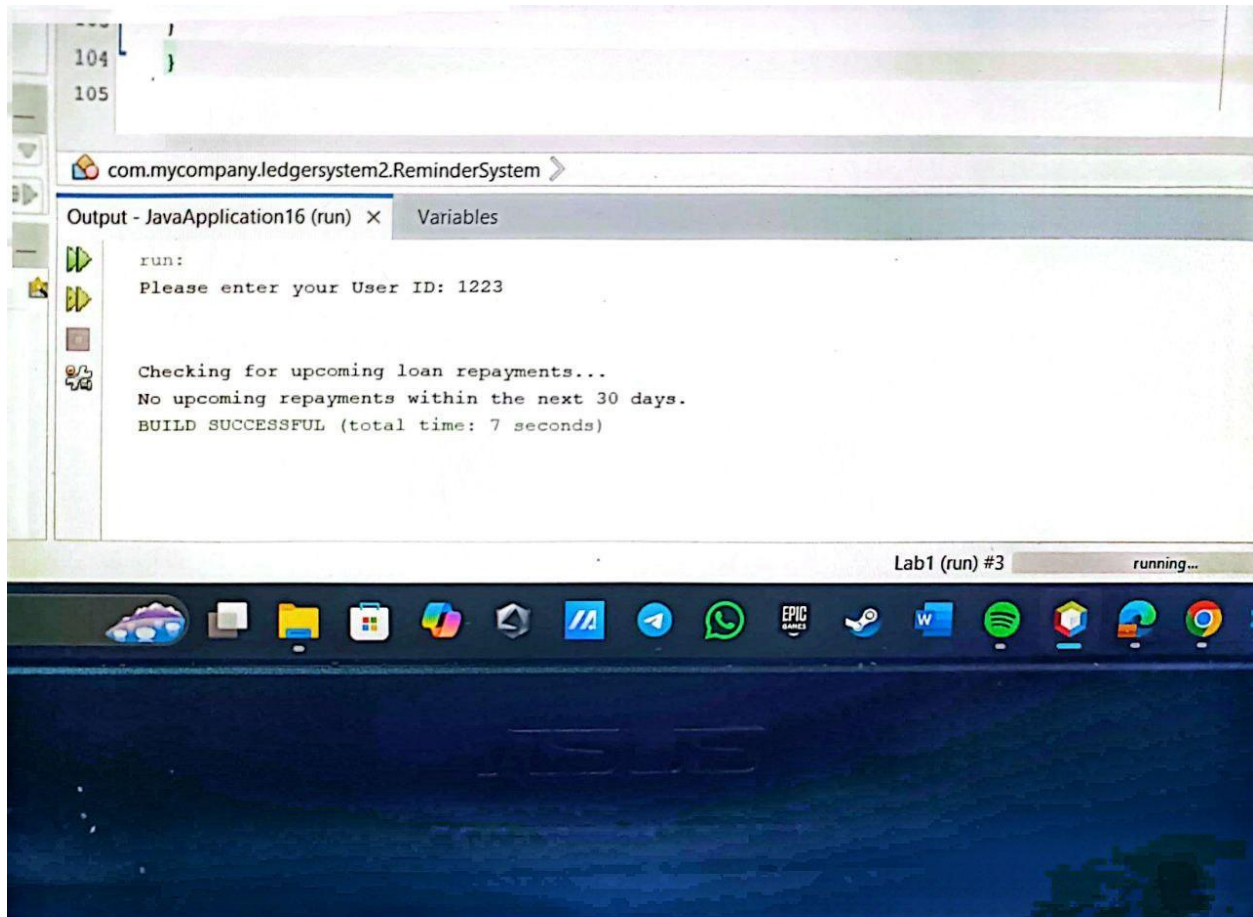
Step 6: Finalizing and Testing

- Test the code with various loan data scenarios, such as:
 - o No loans due in the next 30 days.
 - o Multiple loans due at different times.
 - o Invalid date formats in the CSV file.

4. Flowchart



5. Sample Output



SavingsManager

1. Assigned project

This class manages savings functionalities in the Ledger System. It allows users to activate savings, apply savings to transactions, view savings growth, and transfer savings automatically at the end of each month.

2. Requirements

- **Savings Activation:** Users can activate a savings plan by specifying a percentage of their debit transactions to be saved.
- **Savings Application:** Automatically applies savings from debit transactions and records them persistently.
- **Savings Balance Management:** Displays the current balance of savings and supports auto-transfer to other accounts.
- **Data Validation:** Ensures percentages are within a valid range and transactions are handled securely.
- **Error Handling:** Handles scenarios like invalid input and I/O errors gracefully.

3. Approach to solve the problem

- **Encapsulation of Savings Data:**
 - Stores savings configurations and records in a CSV file (`savings.csv`).
 - Utilizes `Utils.generateId()` for unique IDs.
- **Savings Calculation:**
 - Calculates savings as a percentage of the debit amount.
 - Updates the savings balance after every transaction.
- **Persistent Data Storage:**
 - Uses `BufferedReader` and `BufferedWriter` for data retrieval and storage in CSV files.
- **Monthly Auto-Transfer:**

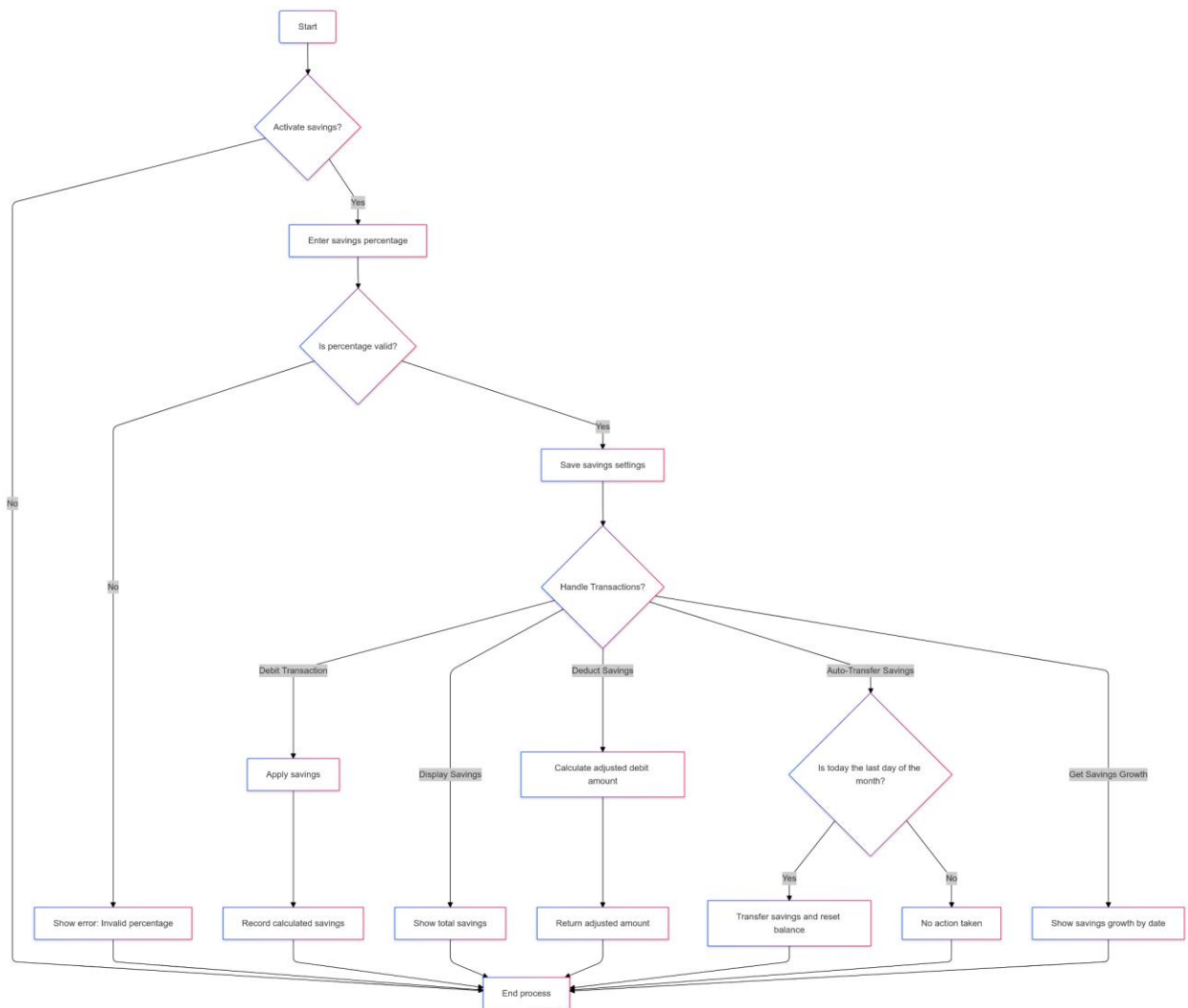
- Transfers accumulated savings at the end of each month using **LocalDate** utilities.

- **Key Methods:**

- **handleSavings:** Activates savings for a user with a specified percentage.
- **applySavings:** Applies the calculated savings to debit transactions.
- **displaySavings:** Shows the current savings balance.
- **autoTransferSavings:** Automatically transfers savings to balance at the month-end.
- **getSavingsGrowth:** Retrieves savings growth data over time.

4. Flow

Chart



5. Sample Output

Main Menu:

1. Record Debit
2. Record Credit
3. Savings
4. View Transaction History
5. Filter & Sort Transaction History
6. Export Transaction History
7. Deposit Interest Predictor
8. Credit Loan
9. Display graph
10. Logout

Choose an option: 3

Activate savings? (Y/N): y

Enter percentage of debit to save (0-100): 50

Savings settings saved successfully!

Welcome, kerol ==

Today is not the end of the month. No action taken.

Current Balance: 2280.00

Current Savings: 0.00

Current Loan: 0.00

Checking for upcoming loan repayments...

No upcoming repayments within the next 30 days.

TransactionManager

1. Assigned project

This class handles all transaction-related functionalities in the Ledger System. It supports recording, displaying, filtering, and exporting transaction history for each user.

2. Requirements

- **Transaction Recording:** Records transactions with details like amount, type (debit/credit), and description.
- **Validation:** Ensures amounts are within set limits, and descriptions adhere to length constraints.
- **Balance Management:** Calculates and displays the current balance based on all transactions.
- **Export Capability:** Allows users to export transaction history into CSV files.
- **Advanced Filtering:** Filters transactions by date range, type, and amount range, and provides sorting options.
- **Error Handling:** Handles invalid inputs and I/O errors during file operations.

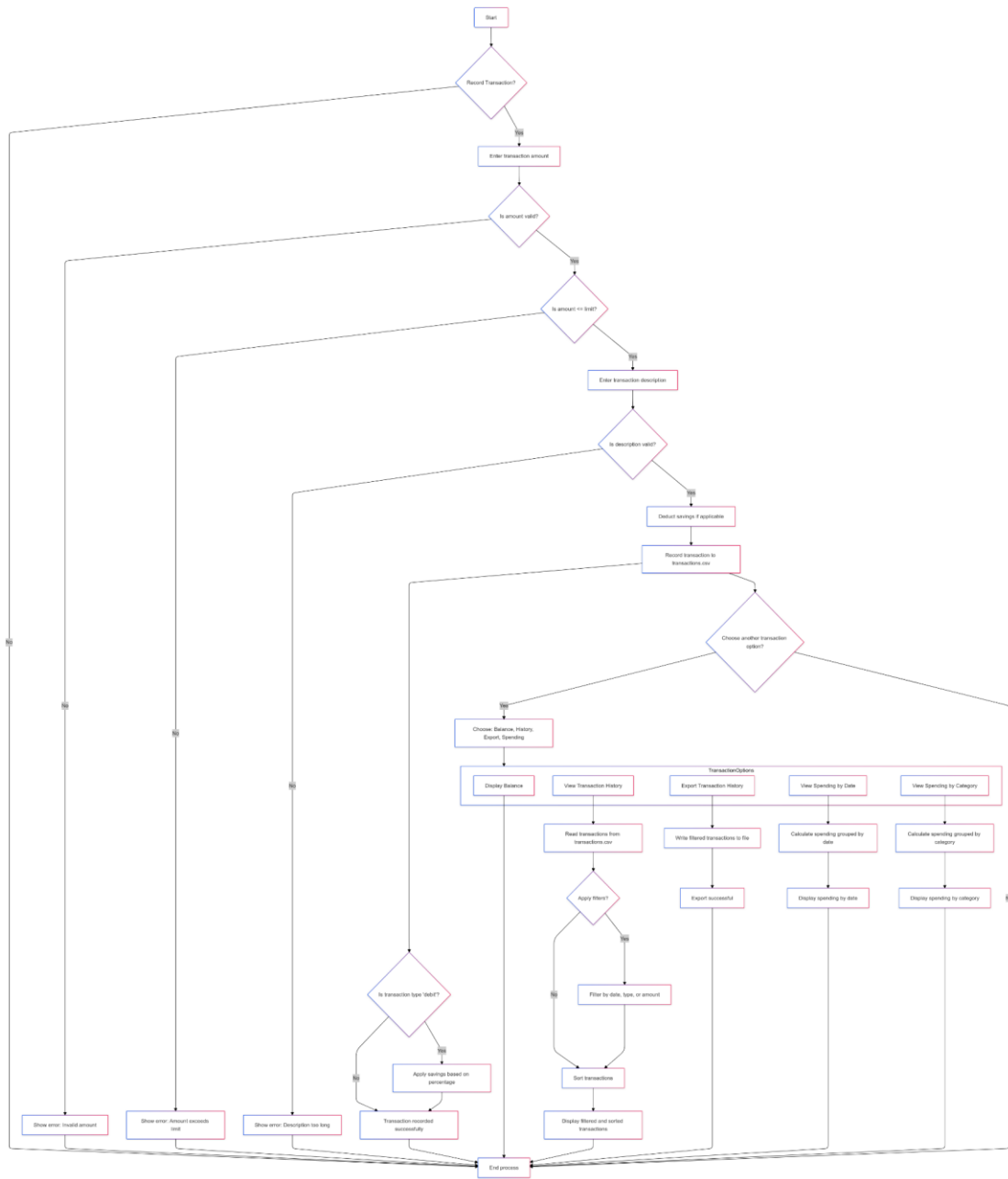
3. Approach to solve the problem

- **Encapsulation of Transaction Data:**
 - Stores transaction data in a CSV file ([transactions.csv](#)).
 - Uses [Utils.generateId\(\)](#) for unique transaction IDs.
- **Validation and Constraints:**
 - Validates input amounts and descriptions before saving transactions.
 - Implements a transaction limit of 10,000 and a description limit of 20 characters.
- **Balance Calculation:**
 - Aggregates transaction amounts to determine the user's current balance.
- **Filtering and Sorting:**
 - Offers multiple options to filter and sort transaction history for better analysis.

- **Key Methods:**

- **recordTransaction:** Records new transactions and applies savings for debit transactions.
- **displayBalance:** Displays the user's current balance.
- **viewTransactionHistory:** Prints all transactions for the user.
- **exportTransactionHistory:** Exports user-specific transaction history into a CSV file.
- **filterAndSortHistory:** Filters and sorts transaction data based on user-specified criteria.

4. Flow Chart



5. Sample Output

Record Debit

```
Main Menu:
1. Record Debit
2. Record Credit
3. Savings
4. View Transaction History
5. Filter & Sort Transaction History
6. Export Transaction History
7. Deposit Interest Predictor
8. Credit Loan
9. Display graph
10. Logout
Choose an option: 1
Transactions are allowed.
Enter amount: 1200
Enter description: beli barang
Debit recorded successfully!

Welcome, kerol ==
Today is not the end of the month. No action taken.
Current Balance: 2280.00
Current Savings: 0.00
Current Loan: 0.00

Checking for upcoming loan repayments...
No upcoming repayments within the next 30 days.
```

Record Credit

```
Main Menu:
1. Record Debit
2. Record Credit
3. Savings
4. View Transaction History
5. Filter & Sort Transaction History
6. Export Transaction History
7. Deposit Interest Predictor
8. Credit Loan
9. Display graph
10. Logout
Choose an option: 2
Transactions are allowed.
Enter amount: 1000
Enter description: gaji
Credit recorded successfully!

Welcome, kerol ==
Today is not the end of the month. No action taken.
Current Balance: 740.00
Current Savings: 3310.00
Current Loan: 0.00

Checking for upcoming loan repayments...
No upcoming repayments within the next 30 days.
```

View Transaction History

Main Menu:

1. Record Debit
2. Record Credit
3. Savings
4. View Transaction History
5. Filter & Sort Transaction History
6. Export Transaction History
7. Deposit Interest Predictor
8. Credit Loan
9. Display graph
10. Logout

Choose an option: 4

Transaction History:

Date: 2025-01-06, Type: debit, Amount: 1200.00, Description: haji

Date: 2025-01-06, Type: credit, Amount: 120.00, Description: beli barang

Date: 2025-01-10, Type: debit, Amount: 1200.00, Description: beli barang

Export Transaction History

Main Menu:

1. Record Debit
2. Record Credit
3. Savings
4. View Transaction History
5. Filter & Sort Transaction History
6. Export Transaction History
7. Deposit Interest Predictor
8. Credit Loan
9. Display graph
10. Logout

Choose an option: 6

Transaction history successfully exported to: user_transactions_1.csv

Welcome, kerol ==

Today is not the end of the month. No action taken.

Current Balance: 740.00

Current Savings: 3310.00

Current Loan: 1010.00

Invalid date format for loan ID: 1

Checking for upcoming loan repayments...

No upcoming repayments within the next 30 days.

Filter and Sort Transaction History

Main Menu:

1. Record Debit
2. Record Credit
3. Savings
4. View Transaction History
5. Filter & Sort Transaction History
6. Export Transaction History
7. Deposit Interest Predictor
8. Credit Loan
9. Display graph
10. Logout

Choose an option: 5

Filter Options:

1. Date Range
2. Transaction Type (Debit/Credit)
3. Amount Range
4. No Filter

Choose a filter option: 2

Enter transaction type (debit/credit): debit

Sorting Options:

1. Date (Newest to Oldest)
2. Date (Oldest to Newest)
3. Amount (Highest to Lowest)
4. Amount (Lowest to Highest)

Choose a sorting option: 2

Filtered and Sorted Transaction History:

Date	Description	Debit	Credit	Balance
2025-01-06	haji	1200.0		1200.00
2025-01-10	beli barang	1200.0		2400.00
2025-01-10	buy	2500.0		4900.00

UserManager

1. Assigned project

- This class is dedicated to managing user-related functionalities in the Ledger System such as registration, login, and user session handling. It ensures that all user interactions with the system are authenticated and secure.

2. Requirements

- **Registration and Login:** Allows users to register by providing their name, email, and password. Ensures secure authentication when users log in.
- **Validation:** Validates user inputs during registration, such as ensuring the email format is correct, passwords meet complexity requirements, and names contain only alphanumeric characters.
- **Password Security:** Likely hashes passwords using a secure hashing algorithm (e.g. bcrypt) before storing them, ensuring passwords are not stored in plaintext.
- **Error Handling:** Handles cases like duplicate emails, incorrect login credentials, or invalid input formats.

3. Approach to solve the problem

❖ Encapsulation of User Data:

- Stores user data (e.g., name, email, hashed password) in a CSV file.
- Uses `Utils.generateId()` to create unique user IDs based on the CSV file's line count.

❖ Secure Authentication:

- Hashes passwords with `BCrypt.hashpw()` during registration.
- Compares hashed passwords using `BCrypt.checkpw()` during login.

❖ Persistent Data Storage:

- Saves user information in `users.csv` using `BufferedWriter`.
- Reads user information during login and name retrieval using `BufferedReader`.

❖ Methods:

➤ registerUser(Scanner scanner):

- Registers a new user by collecting and validating their details.
- Hashes the password and appends the user data to users.csv.

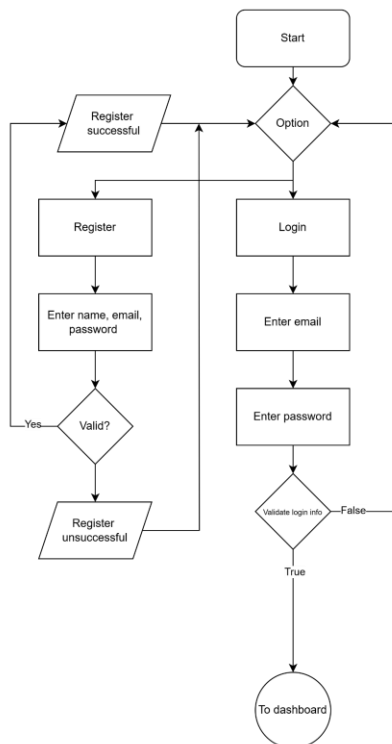
➤ loginUser(Scanner scanner):

- Authenticates a user by matching their email and password from users.csv.
- Returns the user ID if successful, or -1 on failure.

➤ NameUser(int userId):

- Retrieves and returns the name of a user based on their user ID from users.csv.

4. Flow Chart



5. Sample output

```
Welcome to the Ledger System!
1. Register
2. Login
3. Exit
Choose an option: 1
Enter your name: Abu
Enter your email: abu@gmail.com
Enter your password: abul23
Confirm your password: abul23
Registration successful!
```

```
-
Welcome to the Ledger System!
1. Register
2. Login
3. Exit
Choose an option: 2
Enter your email: abu@gmail.com
Enter your password: abul23
Login successful!
```

Utils

1. Assigned project

- This class serves as a utility class, providing common helper methods and reusable functionality for the Ledger System. Its purpose is to centralize code that can be used across different modules, reducing redundancy.

2. Requirements

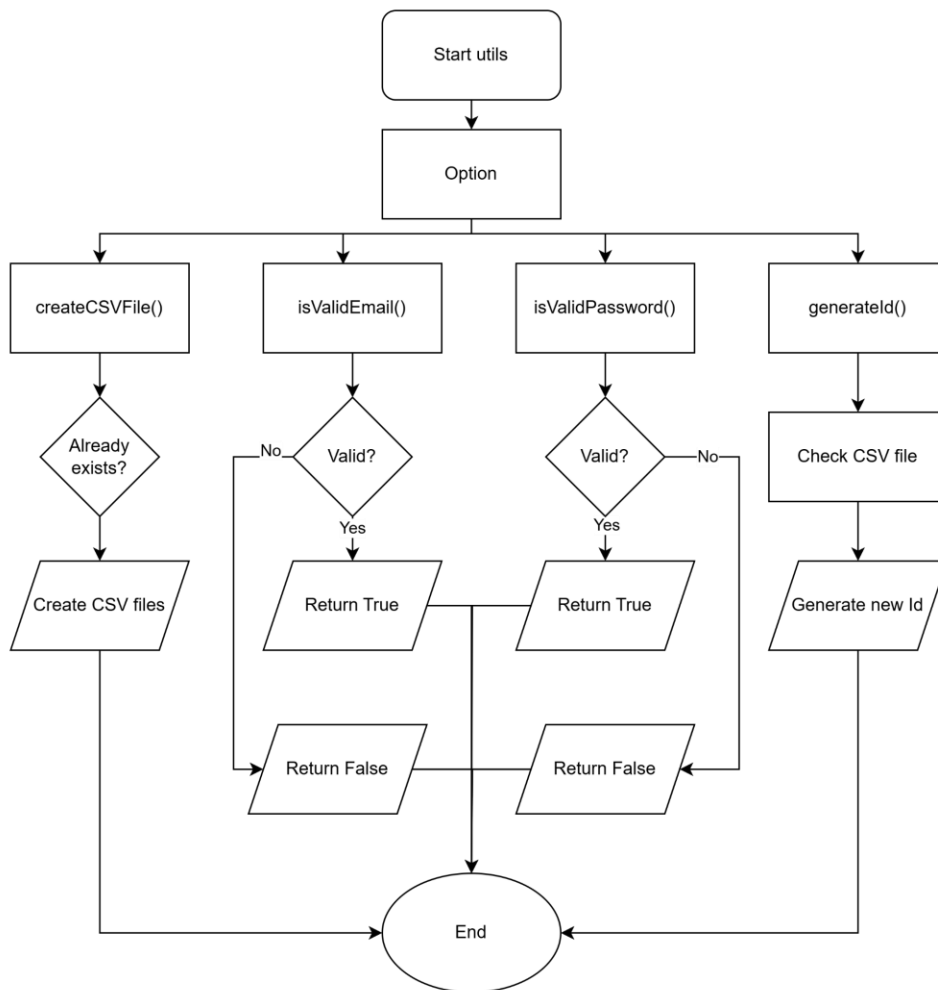
- **Input Validation:** Ensures user inputs, such as email, names, and transaction details, adhere to required formats and constraints.
- **Formatting and Parsing:** Handles operations like formatting dates, parsing strings into numerical values, or generating formatted output.
- **Error Logging:** Provides methods for logging errors or system messages, aiding in debugging and system monitoring.
-

3. Approach to solve the problem

- **File Management:**
 - Uses Java's `java.nio.file` package to check for file existence and create files if missing.
 - Writes file headers with `Files.write()` and appends new content using `StandardOpenOption.CREATE`.
- **Data Consistency:**
 - Uses `BufferedReader` to read CSV files and count existing entries for generating unique IDs.
- **Input Validation:**
 - Implements lightweight validation methods using basic string checks and regular expressions for efficiency and simplicity.
- **Methods:**
 - **ensureCSVFiles():**
 - Creates necessary CSV files for storing data (users, transactions, loans, savings, and repayments).

- Invokes createCSVFile() to set up each file with the appropriate headers.
- **createCSVFile(String filePath, String headers):**
 - Creates a CSV file if it doesn't exist and writes the specified headers.
 - Uses Files.exists() to check for file existence and Files.write() for file creation.
- **generateId(String filePath):**
 - Reads the specified CSV file using BufferedReader and counts the number of lines to generate a new unique ID.
- **isValidEmail(String email):**
 - Validates email strings by ensuring they contain "@" and "." characters.
- **isValidPassword(String password):**
 - Validates password complexity by checking for:
 - Minimum length of 6 characters.
 - Presence of at least one letter and one number using String.matches().

4. Flow Chart



5. Sample output

```
Welcome to the Ledger System!
1. Register
2. Login
3. Exit
Choose an option: 1
Enter your name: ali
Enter your email: aligmail.com
Enter your password: ali
Confirm your password: ali
Invalid email format!
```

```
Welcome to the Ledger System!
1. Register
2. Login
3. Exit
Choose an option: 1
Enter your name: ali
Enter your email: ali@gmail.com
Enter your password: ali
Confirm your password: ali
Password must be at least 6 characters long with letters and numbers.
```